

Smart HAS — Comunidade Conectada

Fase 5 — Capítulo 1: Mobile Hybrid App e a Sociedade 5.0

Relatório de Atividade

Integrante: Yuri Sousa

RM: 556163

FIAP — Faculdade de Informática e Administração Paulista

2026

1. Contextualização

O Smart HAS (também referido como ComunidadeConectada) é um aplicativo híbrido voltado à Sociedade 5.0: conecta pessoas de uma comunidade para compartilhar recursos — ferramentas, itens de saúde, livros, alimentos, eletrônicos — por empréstimo, troca ou doação, com apoio de um assistente de IA (ConectaIA) e visualização geográfica das ofertas.

Nas fases anteriores o projeto foi construído em Flutter (app mobile funcional, porém com dados 100% mockados em memória — sem persistência real, sem autenticação real e sem backend). Esta atividade (Fase 5, Cap. 1) evolui o sistema em três frentes: migração da stack mobile para React Native, criação de um backend real em Spring Boot integrado ao Firebase e criação de um painel administrativo web em Angular consumindo a mesma API. As decisões técnicas desta entrega foram deliberadamente alinhadas ao material didático dos 13 capítulos desta fase (não só ao enunciado da atividade em si), detalhado na seção 2.2.

2. Parte 1 — Escolha da Stack Tecnológica e Justificativa

2.1 Decisão

Optou-se por migrar o app mobile de Flutter/Dart para React Native (Opção A do enunciado), reescrevendo as telas principais e consumindo a mesma API REST do backend.

2.2 Justificativa técnica

- O material da Fase 5 dedica 5 dos seus 13 capítulos inteiramente a React Native (estrutura de projeto e padrão MVC por tela, estilização com Flexbox e react-native-elements, geolocalização e persistência, e um capítulo final que integra Hooks, mapas, câmera e notificações push) — é, disparado, o maior bloco de conteúdo da fase. Migrar para React Native é a forma de a entrega efetivamente demonstrar esse conteúdo, em vez de deixá-lo só na teoria.
- O enunciado da Opção A pede explicitamente componentes funcionais com View, Text, Image, Button e navegação via React Navigation — todos usados diretamente na migração.
- O ambiente já contava com Node.js funcional, permitindo montar o projeto com Expo (abordagem discutida no Cap. 02 do material como caminho mais rápido para um MVP) e testar de verdade contra a API real, em vez de escrever código sem qualquer verificação.
- A migração seguiu os padrões ensinados: componentes funcionais com Hooks (useState/useEffect, conforme o Cap. 06 recomenda como abordagem atual, em vez de componentes de classe), React Navigation (Stack + Tabs) para a navegação entre as 10 telas, StyleSheet + Flexbox para o layout, e AsyncStorage para persistir o token de autenticação (padrão do Cap. 05).

2.3 Verificação realizada e uma limitação de ambiente

O plano original era rodar a aplicação em um emulador Android real (AVD "Pixel_7", já disponível no ambiente) via `expo run:android`. O emulador travou e, em seguida, encerrou de forma inesperada (crash confirmado por dump gerado pelo Crashpad) — causa identificada: o driver de kernel do anti-cheat Vanguard (Riot Games), instalado na máquina, é conhecidamente incompatível com o hypervisor usado pelo emulador Android no Windows. Desativar o Vanguard não bastou, pois o driver permanece carregado em memória até a próxima reinicialização do sistema — reinicialização que não fazia sentido exigir do usuário só para esta verificação.

Diante disso, a verificação foi feita via `expo start --web` (react-native-web), testando a aplicação real no navegador contra o backend rodando de verdade: login com usuário semeado, listagem dos recursos vindos da API, abertura de um chat (que envia a mensagem inicial de interesse e recebe a resposta persistida no backend), bloqueio correto de ações para o modo Visitante, formulário de oferta com os seletores de categoria/tipo/condição, o mapa interativo e o assistente ConectaIA (detalhados a seguir). A única funcionalidade que realmente depende de hardware nativo (câmera/galeria via expo-image-picker, usada para a foto de perfil e do recurso) não pôde ser exercitada nesse ambiente web, ficando validada apenas por leitura de código; notificações push ficam documentadas como próximo passo (dependem de credenciais Apple/Google que o projeto não possui).

- Mapa: em vez de react-native-maps (que exigiria uma chave paga do Google Maps para renderizar tiles no Android), o mapa da comunidade usa Leaflet + OpenStreetMap dentro de um WebView — gratuito,

sem chave, e com o mesmo HTML rodando tanto no WebView nativo (iOS/Android) quanto num <iframe> na versão web (arquivo MapWebView.web.tsx), o que permitiu testar o mapa de verdade, com pan/zoom e pins clicáveis mostrando categoria e ofertante, mesmo sem emulador.

- ConectaIA: chama a API do Gemini (Google AI Studio) diretamente do cliente, com chave própria configurada via variável de ambiente. Testado com uma pergunta real enviada pelo chat e resposta recebida da IA de verdade — não é mais um placeholder.

2.4 Roadmap tecnológico

Concluído nas fases anteriores:

- Fase 1-4: definição de escopo, arquitetura, protótipo funcional em Flutter com dados mockados.

Concluído nesta atividade (Fase 5, Cap. 1):

- App mobile migrado para React Native (Expo + TypeScript), reproduzindo as 10 telas do app Flutter original, agora consumindo a API real.
- Backend Spring Boot completo com autenticação (Firebase Authentication) e persistência real (Firestore), CRUD de usuários, recursos e mensagens.
- Painel administrativo em Angular consumindo a mesma API REST, com autenticação, estatísticas, e CRUD de usuários/recursos.

Planejado para as próximas fases:

- Testar a build nativa em um dispositivo físico ou em uma máquina sem o conflito do Vanguard, para validar câmera, notificações e navegação 100% nativa (o mapa e a IA já funcionam de verdade também na versão web, então essa parte não depende do build nativo).
- Upload real de imagens (hoje o campo imageUrl aceita apenas texto/URL — não há endpoint multipart no backend).
- Notificações push reais via Firebase Cloud Messaging (Cap. 06 do material), reaproveitando a base de usuários já em Firebase Authentication.
- Sessão persistente no app mobile (login automático ao reabrir o app usando o token já salvo).
- Modelo de conversas do backend hoje agrupa mensagens por par de usuários; reconciliar com a visão "uma conversa por recurso" da tela de chat do app.
- Camada de IA Logistics Extension mencionada no contexto geral do projeto — roteamento inteligente de ofertas/pedidos por proximidade, usando o histórico agora persistido no backend.
- Regras de segurança do Firestore hoje em modo teste (leitura/escrita liberada) — restringir por usuário autenticado antes de qualquer uso além do acadêmico, seguindo o alerta do próprio Cap. 13 do material sobre esse ponto.
- Habilitar HTTPS/TLS na comunicação mobile/web ↔ backend (hoje em HTTP simples, adequado para desenvolvimento local, mas não para produção).

3. Parte 2 — Back-end Escalável com Spring Boot

Backend implementado em Java 21 / Spring Boot 4.1.0, no diretório backend/smarthas-api, seguindo a especificação em API_CONTRACT.md. Camada de persistência migrada de H2 (relacional local) para Firebase — Firestore para dados e Firebase Authentication para identidade — alinhando a entrega ao Cap. 13 do material da fase, um tutorial dedicado inteiramente a Firebase Authentication + Realtime Database, e à sugestão explícita do enunciado ("banco de dados de forma apropriada — Firebase, por exemplo").

3.1 Arquitetura

- Camadas: Controller → Service → Repository, com injeção via construtor — os "repositórios" agora encapsulam chamadas ao Firestore via Firebase Admin SDK em vez de Spring Data JPA.
- Entidades: User, Resource, Message, com enums para papel (USER/ADMIN), categoria, condição, tipo e disponibilidade do recurso. IDs deixaram de ser sequenciais (Long) e passaram a ser Strings opacas — UID do Firebase Authentication para usuários, ID de documento do Firestore para recursos e mensagens.

- DTOs (records) para requests/responses — a senha nunca é persistida nem serializada pelo backend: quem guarda e valida a senha é o próprio Firebase Authentication.
- Autenticação: cadastro cria a conta no Firebase Authentication (Admin SDK) e o perfil no Firestore; login verifica a senha chamando a API REST Identity Toolkit do Firebase (o Admin SDK não expõe verificação de senha) e, uma vez validada, o backend emite seu próprio JWT (biblioteca jjwt) com o UID do Firebase como sujeito — o contrato de autenticação exposto aos clientes (Bearer token) não mudou em nada.
- Autorização: rotas públicas (listagem/detalhe de recursos, autenticação), rotas autenticadas (perfil, mensagens, publicar recurso) e rotas exclusivas de ADMIN (listar/excluir usuários, estatísticas) — inalteradas em relação à versão anterior.
- Comunicação mobile/web ↔ backend: protegida por token JWT em toda rota autenticada (o "segredo" nunca trafega, só o token assinado) e CORS restrito às origens conhecidas. A camada de transporte roda em HTTP simples nesta entrega (ambiente local de desenvolvimento, sem certificado); habilitar HTTPS/TLS é item de roadmap para um deploy real (seção 2.4).
- Persistência: Firestore (NoSQL, documentos), com filtros de categoria/tipo/disponibilidade delegados ao Firestore e busca textual ("q") + paginação resolvidas em memória no backend, já que o Firestore não tem busca full-text nativa.
- Tratamento de erros: `@RestControllerAdvice` padronizando respostas de erro (400 validação, 401 não autenticado, 403 proibido, 404 não encontrado, 409 conflito) no formato `{ timestamp, status, error, message, path }` — mesmo formato de antes.
- Documentação: springdoc-openapi — Swagger UI em `/swagger-ui/index.html`.
- Dados de exemplo (seed): reaproveita os mesmos usuários/recursos que existiam mockados no app Flutter da Fase 4 (Yuri Ribeiro, João Silva, Maria Oliveira, Carlos Souza; Furadeira Bosch, Cadeira de Rodas, Livro Dom Casmurro), agora persistidos de verdade no Firebase.

3.2 Principais endpoints

Método	Rota	Descrição
POST	<code>/api/auth/register</code>	Cria conta e retorna token JWT
POST	<code>/api/auth/login</code>	Autentica e retorna token JWT
GET	<code>/api/users/me</code>	Dados do usuário autenticado
PUT	<code>/api/users/me</code>	Atualiza perfil
GET	<code>/api/users</code>	Lista usuários (paginado, somente ADMIN)
GET	<code>/api/resources</code>	Lista recursos (público, paginado, com filtros)
POST	<code>/api/resources</code>	Publica um novo recurso (autenticado)
PUT/ DELETE	<code>/api/resources/{id}</code>	Edita/remove recurso (dono ou ADMIN)
GET	<code>/api/messages/conversation/{userId}</code>	Histórico de mensagens com um usuário
POST	<code>/api/messages</code>	Envia mensagem
GET	<code>/api/admin/stats</code>	Estatísticas gerais (somente ADMIN)

3.3 Verificação realizada

O backend foi de fato compilado e executado localmente (`./mvnw spring-boot:run`) e testado ponta a ponta via curl, nas duas versões (H2 e, depois, Firebase): cadastro, login, listagem/criação/edição/remoção de recursos (com checagem de permissão dono-vs-admin), rotas autenticadas (`/me`, `/mine`), estatísticas administrativas, envio/listagem de mensagens, e os cenários de erro (e-mail duplicado → 409, dados inválidos → 400, recurso inexistente → 404). Na versão com Firebase, o teste incluiu cadastrar um usuário novo e depois logar com exatamente aquelas credenciais em uma chamada independente — prova de que a conta foi criada de verdade no Firebase Authentication, não simulada.

Um bug real foi encontrado e corrigido durante essa verificação: o SDK do Firebase Admin falhava de forma consistente ("Not in GZIP format") ao chamar a API de autenticação, aparentemente por alguma camada de rede/antivírus local corrompendo a resposta comprimida. A correção foi contornar o SDK nesse ponto específico, fazendo as chamadas de criação/consulta de usuário via REST direto (com um token OAuth2 obtido a partir da própria credencial de serviço), mantendo o Firestore via SDK normalmente.

4. Parte 3 — Integração Web com Angular

Painel administrativo em Angular (standalone components), no diretório web/smarthas-admin, consumindo a mesma API REST do app mobile.

4.1 O que foi implementado

- Rotas: /home (pública), /login e /admin (protegida por AuthGuard).
- AuthService com login via HttpClient, token JWT salvo em localStorage, e um HttpInterceptor que anexa o header Authorization automaticamente em toda chamada.
- Painel /admin: cartões de estatísticas (GET /api/admin/stats), quebra de recursos por categoria, tabela de usuários com exclusão, tabela de recursos com exclusão, e um formulário de criação de recurso.
- Data binding demonstrado nas quatro formas exigidas: interpolação {{ }} (ex.: {{ stats.totalUsers }}), property binding [] (ex.: [disabled]), event binding () (ex.: (click), (ngSubmit)) e two-way binding [()] com [(ngModel)] no formulário de login e no formulário de criação de recurso.
- Diretivas estruturais *ngIf (estados de carregamento/erro/vazio, badge de verificado) e *ngFor (linhas das tabelas, opções de categoria) usadas explicitamente conforme pedido no enunciado.

4.2 Verificação realizada

O projeto foi compilado com sucesso (ng build) e executado com ng serve na porta 4200. Com o backend rodando na porta 8080, o login administrativo (admin@smarthas.com) e a navegação pelo painel foram testados ao vivo no navegador: os dados exibidos (usuários, recursos, estatísticas) refletem exatamente o estado real do backend, incluindo alterações feitas durante os próprios testes do backend (ex.: um recurso renomeado via API apareceu atualizado no painel). Após a migração do backend para Firebase, as interfaces TypeScript (id: number → id: string) foram atualizadas e o build reconferido sem erros.

5. Como executar o projeto

5.1 Backend

```
cd backend/smarthas-api
./mvnw spring-boot:run
```

Sobe em http://localhost:8080. Requer o arquivo config/firebase-service-account.json (credencial do projeto Firebase, não incluído no controle de versão) e a Web API Key do projeto configurada em application.properties. Credenciais seed: admin@smarthas.com / admin123 (ADMIN) e yuri@exemplo.com / 123456 (usuário comum). Swagger: http://localhost:8080/swagger-ui/index.html.

5.2 Painel Web (Angular)

```
cd web/smarthas-admin
npm install
npx ng serve
```

Sobe em http://localhost:4200. Faça login em /login com a conta admin.

5.3 App Mobile (React Native)

```
cd mobile-react-native/SmartHASApp
npm install
npx expo start --web
```

Abre a versão web (react-native-web) no navegador — caminho usado para verificação neste ambiente devido ao conflito do Vanguard com o emulador Android (ver seção 2.3). Em uma máquina sem esse conflito, prefira `npx expo run:android` para rodar nativamente em um emulador ou dispositivo. O app resolve automaticamente o endereço do backend (10.0.2.2:8080 no emulador Android, localhost:8080 nas demais plataformas).

5.4 App Mobile (Flutter — versão anterior, mantida como referência)

```
cd mobile/FlutterProject
flutter pub get
flutter run
```

Versão em Flutter da Fase 4/início da Fase 5, já integrada ao backend real, mantida no repositório como histórico do projeto — a entrega desta atividade usa a versão React Native (seção 5.3).

6. Entregáveis desta atividade

- Código-fonte completo (mobile-react-native/, mobile/, backend/, web/) no repositório GitHub: https://github.com/YuriRASousa/ComunidadeConectada_FASE5
- Apresentação de slides (PDF) — arquivo em anexo.
- Vídeo de demonstração (até 5 min, YouTube não listado): <https://youtu.be/6GpY25Bgr6E>

7. Conclusão

Esta etapa transformou o Smart HAS de um protótipo mobile com dados mockados em um sistema full-stack real e alinhado ao conteúdo da fase: app React Native, backend Spring Boot com autenticação e persistência via Firebase (Authentication + Firestore), e um painel administrativo Angular, todos integrados pelo mesmo contrato de API. O aprendizado prático cobriu React Native com Hooks e React Navigation, autenticação via Firebase, modelagem de API REST sobre um banco NoSQL, tratamento de erros padronizado, consumo de API em três clientes diferentes, e depuração ponta a ponta entre aplicações distintas — incluindo diagnosticar e contornar problemas reais de ambiente (conflito do Vanguard com o emulador Android, falha de transporte HTTP do SDK do Firebase) — base sólida para as próximas fases do curso.